

## Section 79 Editorial — Directed Graph Decomposition

### A. Condensation Profile

#### How to find it

Contract mutual-reachability classes; the quotient is a DAG. Scan only inter-component edges to mark sources and sinks.

#### Correctness

SCCs partition the vertices by mutual reachability. Any directed cycle between contracted classes would make all classes on it mutually reachable, a contradiction, so the quotient is a DAG. The final scan marks a class precisely when an original edge crosses its boundary; therefore the unmarked indegrees and outdegrees are exactly those requested.

#### Complexity

$O(n+m)$  time and memory.

#### C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
```

```

        st.pop_back();
        for (int v : rg[u])
            if (id[v] < 0)
                id[v] = c, st.push_back(v);
    }
    c++;
}
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;

```

```

for (int s = 0; s < n; s++)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (auto [v, k] : l.g[u])
                if (!l.bridge[k] && id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }

    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

```

```

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
                sort(vs.begin(), vs.end());
                vs.erase(unique(vs.begin(), vs.end()), vs.end());
                int block = t.size();
                t.push_back({});
                for (int x : vs)
                    t[block].push_back(x), t[x].push_back(block);
            }
        }
    }
    return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();

```

```

    st.pop_back();
    for (int v : g[u])
        if (!reach[v])
            reach[v] = 1, st.push_back(v);
}
vector<bitset<1500>> d(n, reach);
d[0].reset();
d[0][0] = 1;
bool change = 1;
while (change) {
    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
}
return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    SCC s(n, e);
    vector<char> in(s.c), out(s.c);
    for (auto [u, v] : e)
        if (s.id[u] != s.id[v])
            out[s.id[u]] = in[s.id[v]] = 1;
    cout << s.c << ' ' << count(in.begin(), in.end(), 0) << ' '
        << count(out.begin(), out.end(), 0) << '\n';
}

```

## Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True

```

```

while stack:
    node, index = stack[-1]
    if index < len(graph[node]):
        target = graph[node][index]
        stack[-1] = (node, index + 1)
        if not seen[target]:
            seen[target] = True
            stack.append((target, 0))
    else:
        order.append(node)
        stack.pop()
component = [-1] * n
count = 0
for start in reversed(order):
    if component[start] != -1:
        continue
    component[start] = count
    stack = [start]
    while stack:
        node = stack.pop()
        for target in reverse[node]:
            if component[target] == -1:
                component[target] = count
                stack.append(target)
    count += 1
return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])
                if low[target] > tin[node]:
                    bridges[edge_id] = True
                if low[target] >= tin[node]:
                    separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

```

```

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):
            if up[level][a] != up[level][b]:
                a, b = up[level][a], up[level][b]
        return up[0][a]

    def distance(a, b):
        ancestor = lca(a, b)
        return depth[a] + depth[b] - 2 * depth[ancestor]

    return distance

```

```

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):

```

```

        if not (reachable >> node & 1):
            continue
        value = reachable
        for parent in predecessors[node]:
            if reachable >> parent & 1:
                value &= dom[parent]
        value |= 1 << node
        if value != dom[node]:
            dom[node] = value
            changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    component, count = scc(n, edges)
    indegree, outdegree = [0] * count, [0] * count
    for u, v in edges:
        if component[u] != component[v]:
            outdegree[component[u]] = 1
            indegree[component[v]] = 1
    print(count, indegree.count(0), outdegree.count(0))

if __name__ == "__main__":
    main()

```

## B. Clause Satisfiability

### How to find it

Translate every clause into two contraposed implications. Unsatisfiability is exactly collision of a literal with its negation inside one SCC.

### Correctness

If a literal and its negation share an SCC, each implies the other, so either truth choice forces a contradiction. If no pair collides, take SCCs in reverse topological order and set an unassigned component false and its complementary component true. Every implication goes forward in this order, so no true premise can have a false conclusion; hence every clause holds.

### Complexity

$O(n+m)$  time and memory.

### C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
    }
};

```

```

for (int s = 0; s < n; s++)
    if (!seen[s]) {
        vector<pair<int, int>> st = {{s, 0}};
        seen[s] = 1;
        while (!st.empty()) {
            auto &[u, i] = st.back();
            if (i < (int)g[u].size()) {
                int v = g[u][i++];
                if (!seen[v])
                    seen[v] = 1, st.push_back({v, 0});
            } else
                order.push_back(u), st.pop_back();
        }
    }
reverse(order.begin(), order.end());
for (int s : order)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : rg[u])
                if (id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
}
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {

```

```

        st.pop_back();
        int id = parent_edge[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        sub[p] += sub[u];
        low[p] = min(low[p], low[u]);
        if (low[u] > tin[p])
            bridge[id] = 1;
        if (low[u] >= tin[p])
            parts[p].push_back(sub[u]);
    }
}
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)

```

```

        up[j][i] = up[j - 1][up[j - 1][i]];
    }
    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

vector<vector<int>>> block_cut(int n, const vector<pair<int, int>>> &e) {
    vector<vector<pair<int, int>>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
            }
        }
    }
}

```

```

    }
    sort(vs.begin(), vs.end());
    vs.erase(unique(vs.begin(), vs.end()), vs.end());
    int block = t.size();
    t.push_back({});
    for (int x : vs)
        t[block].push_back(x), t[x].push_back(block);
    }
}
}
return t;
}
vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {
        change = 0;
        for (int v = 1; v < n; v++)
            if (reach[v]) {
                bitset<1500> x = reach;
                for (int p : pred[v])
                    if (reach[p])
                        x &= d[p];
                x[v] = 1;
                if (x != d[v])
                    d[v] = x, change = 1;
            }
    }
    return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e;
    auto node = [](int x) { return 2 * (abs(x) - 1) + (x < 0); };
    while (m--) {
        int x, y;
        cin >> x >> y;
        int a = node(x), b = node(y);
        e.push_back({a ^ 1, b});
        e.push_back({b ^ 1, a});
    }
    SCC s(2 * n, e);
    for (int i = 0; i < n; i++)
        if (s.id[2 * i] == s.id[2 * i + 1]) {
            cout << "NO\n";

```

```

    return 0;
}
cout << "YES\n";
}

```

## Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n

```

```

bridges = [False] * len(edges)
separating = [[] for _ in range(n)]
timer = 0

def dfs(node, parent_edge=-1):
    nonlocal timer
    tin[node] = low[node] = timer
    timer += 1
    subtree[node] = 1
    for target, edge_id in graph[node]:
        if edge_id == parent_edge:
            continue
        if tin[target] != -1:
            low[node] = min(low[node], tin[target])
        else:
            dfs(target, edge_id)
            subtree[node] += subtree[target]
            low[node] = min(low[node], low[target])
            if low[target] > tin[node]:
                bridges[edge_id] = True
            if low[target] >= tin[node]:
                separating[node].append(subtree[target])

dfs(0)
return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:

```

```

        if target != parent:
            depth[target] = depth[node] + 1
            order.append(target)
            stack.append((target, node))
for level in range(1, levels):
    previous = up[level - 1]
    up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)

```

```

        elif tin[target] < tin[node]:
            edge_stack.append(edge_id)
            low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = []

    def node(x):
        return 2 * (abs(x) - 1) + (x < 0)

    for i in range(2, 2 + 2 * m, 2):
        a, b = node(data[i]), node(data[i + 1])
        edges.extend(((a ^ 1, b), (b ^ 1, a)))
    component, _ = scc(2 * n, edges)
    print(
        "YES"
        if all(component[2 * i] != component[2 * i + 1] for i in range(n))
        else "NO"
    )

if __name__ == "__main__":
    main()

```

## C. Lexicographic Euler Trail

### How to find it

Check degree imbalance, consume sorted outgoing edges with Hierholzer, and require exactly  $m+1$  output vertices.

### Correctness

The degree conditions follow by counting arrivals and departures of a trail. Hierholzer appends a vertex only when it has no unused outgoing edge, so reversing the appended list makes consecutive vertices endpoints of consumed edges and uses every consumed edge once. A list of length  $m+1$  proves all edges were consumed; otherwise the edge set was disconnected from the start. Ordered adjacency gives the first differing feasible edge the smallest endpoint.

### Complexity

$O(n+m)$  time and memory.

### C++

```
#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
        reverse(order.begin(), order.end());
        for (int s : order)
            if (id[s] < 0) {
                id[s] = c;
                vector<int> st = {s};
                while (!st.empty()) {
                    int u = st.back();
                    st.pop_back();
                    for (int v : rg[u])
                        if (id[v] < 0)
                            id[v] = c, st.push_back(v);
                }
                c++;
            }
    }
};
struct LowLink {
```

```

int n, timer = 0;
vector<pair<int, int>> e;
vector<vector<pair<int, int>>> g;
vector<int> tin, low, sub, parent_edge, it;
vector<char> bridge;
vector<vector<int>> parts;
LowLink(int n, vector<pair<int, int>> e)
    : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
      bridge(e.size()), parts(n) {
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    tin[0] = low[0] = timer++;
    sub[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent_edge[u])
                continue;
            if (tin[v] >= 0)
                low[u] = min(low[u], tin[v]);
            else {
                parent_edge[v] = id;
                tin[v] = low[v] = timer++;
                sub[v] = 1;
                st.push_back(v);
            }
        } else {
            st.pop_back();
            int id = parent_edge[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            sub[p] += sub[u];
            low[p] = min(low[p], low[u]);
            if (low[u] > tin[p])
                bridge[id] = 1;
            if (low[u] >= tin[p])
                parts[p].push_back(sub[u]);
        }
    }
};

pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    return {id, l.parts};
}

```

```

    }
    c++;
}
vector<vector<int>> t(c);
for (int i = 0; i < (int)e.size(); i++)
    if (l.bridge[i]) {
        auto [u, v] = e[i];
        u = id[u];
        v = id[v];
        t[u].push_back(v);
        t[v].push_back(u);
    }
return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }
    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
        return up[0][a];
    }
    int dist(int a, int b) {
        int c = lca(a, b);
        return d[a] + d[b] - 2 * d[c];
    }
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;

```

```

tin[0] = low[0] = timer++;
vector<int> dfs = {0};
while (!dfs.empty()) {
    int u = dfs.back();
    if (it[u] < (int)g[u].size()) {
        auto [v, id] = g[u][it[u]++];
        if (id == parent[u])
            continue;
        if (tin[v] < 0) {
            parent[v] = id;
            edges.push_back(id);
            tin[v] = low[v] = timer++;
            dfs.push_back(v);
        } else if (tin[v] < tin[u])
            edges.push_back(id), low[u] = min(low[u], tin[v]);
    } else {
        dfs.pop_back();
        int id = parent[u];
        if (id < 0)
            continue;
        auto [a, b] = e[id];
        int p = a ^ b ^ u;
        low[p] = min(low[p], low[u]);
        if (low[u] >= tin[p]) {
            vector<int> vs;
            while (1) {
                int x = edges.back();
                edges.pop_back();
                vs.push_back(e[x].first);
                vs.push_back(e[x].second);
                if (x == id)
                    break;
            }
            sort(vs.begin(), vs.end());
            vs.erase(unique(vs.begin(), vs.end()), vs.end());
            int block = t.size();
            t.push_back({});
            for (int x : vs)
                t[block].push_back(x), t[x].push_back(block);
        }
    }
}
return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);
    for (auto [u, v] : e)
        g[u].push_back(v), pred[v].push_back(u);
    bitset<1500> reach;
    reach[0] = 1;
    vector<int> st = {0};
    while (!st.empty()) {
        int u = st.back();
        st.pop_back();
        for (int v : g[u])
            if (!reach[v])
                reach[v] = 1, st.push_back(v);
    }
    vector<bitset<1500>> d(n, reach);
    d[0].reset();
    d[0][0] = 1;
    bool change = 1;
    while (change) {

```

```

    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
    }
    return d;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n);
    vector<int> in(n), out(n);
    for (int i = 0, u, v; i < m; i++)
        cin >> u >> v, --u, --v, g[u].push_back(v), out[u]++, in[v]++;
    if (!m) {
        cout << 1 << '\n';
        return 0;
    }
    vector<int> plus, minus;
    for (int v = 0; v < n; v++) {
        if (out[v] - in[v] == 1)
            plus.push_back(v);
        else if (in[v] - out[v] == 1)
            minus.push_back(v);
        else if (in[v] != out[v]) {
            cout << "IMPOSSIBLE\n";
            return 0;
        }
    }
    if (!(plus.empty() && minus.empty() && out[0]) ||
        (plus == vector<int>{0} && minus.size() == 1)) {
        cout << "IMPOSSIBLE\n";
        return 0;
    }
    for (auto &v : g)
        sort(v.rbegin(), v.rend());
    vector<int> st = {0}, ans;
    while (!st.empty()) {
        int u = st.back();
        if (g[u].empty())
            ans.push_back(u), st.pop_back();
        else {
            int v = g[u].back();
            g[u].pop_back();
            st.push_back(v);
        }
    }
    if ((int)ans.size() != m + 1)
        cout << "IMPOSSIBLE\n";
    else {
        reverse(ans.begin(), ans.end());
        for (int i = 0; i <= m; i++)
            cout << ans[i] + 1 << " \n"[i == m];
    }
}

```

```
}
```

## Python

```
import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        reverse[v].append(u)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        stack = [(start, 0)]
        seen[start] = True
        while stack:
            node, index = stack[-1]
            if index < len(graph[node]):
                target = graph[node][index]
                stack[-1] = (node, index + 1)
                if not seen[target]:
                    seen[target] = True
                    stack.append((target, 0))
            else:
                order.append(node)
                stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0
```

```

def dfs(node, parent_edge=-1):
    nonlocal timer
    tin[node] = low[node] = timer
    timer += 1
    subtree[node] = 1
    for target, edge_id in graph[node]:
        if edge_id == parent_edge:
            continue
        if tin[target] != -1:
            low[node] = min(low[node], tin[target])
        else:
            dfs(target, edge_id)
            subtree[node] += subtree[target]
            low[node] = min(low[node], low[target])
            if low[target] > tin[node]:
                bridges[edge_id] = True
            if low[target] >= tin[node]:
                separating[node].append(subtree[target])

dfs(0)
return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))

```

```

for level in range(1, levels):
    previous = up[level - 1]
    up[level] = [previous[previous[node]] for node in range(size)]

def lca(a, b):
    if depth[a] < depth[b]:
        a, b = b, a
    difference = depth[a] - depth[b]
    for level in range(levels):
        if difference >> level & 1:
            a = up[level][a]
    if a == b:
        return a
    for level in range(levels - 1, -1, -1):
        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
    return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

```

```

dfs(0)
return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()
        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    graph = [[] for _ in range(n)]
    indeg = [0] * n
    out = [0] * n
    for i in range(2, 2 + 2 * m, 2):
        u, v = data[i] - 1, data[i + 1] - 1
        graph[u].append(v)
        out[u] += 1
        indeg[v] += 1
    for row in graph:
        row.sort(reverse=True)
    if m == 0:
        print(1)
        return
    plus = [v for v in range(n) if out[v] - indeg[v] == 1]
    minus = [v for v in range(n) if indeg[v] - out[v] == 1]
    valid = all(abs(out[v] - indeg[v]) <= 1 for v in range(n)) and (
        (not plus and not minus and out[0])
        or (plus == [0] and len(minus) == 1)
    )
    if not valid:
        print("IMPOSSIBLE")
        return
    stack = [0]

```

```

path = []
while stack:
    if graph[stack[-1]]:
        stack.append(graph[stack[-1]].pop())
    else:
        path.append(stack.pop())
if len(path) != m + 1:
    print("IMPOSSIBLE")
else:
    print(*(x + 1 for x in reversed(path)))

if __name__ == "__main__":
    main()

```

## D. Unavoidable Checkpoints

### How to find it

Start with every reachable vertex as a possible dominator, intersect predecessor sets to a fixed point, and inspect the target's set.

### Correctness

The root dominates only itself initially. For any other reachable vertex, a checkpoint dominates it exactly when it dominates every immediate predecessor, except the vertex itself which always dominates itself. Thus the fixed-point equation is necessary. Starting from the universal reachable set and repeatedly intersecting can only delete false candidates and converges to the greatest solution, which is the actual family of all-path dominators.

### Complexity

At most  $O(n(n+m))$  set-intersection work under  $n \leq 1500$ ; bit-parallel storage uses  $O(n^2 / \text{word\_size})$  machine words.

### C++

```

#include <bits/stdc++.h>
using namespace std;
struct SCC {
    int n, c = 0;
    vector<vector<int>> g, rg;
    vector<int> id, order;
    SCC(int n, vector<pair<int, int>> e) : n(n), g(n), rg(n), id(n, -1) {
        for (auto [u, v] : e)
            g[u].push_back(v), rg[v].push_back(u);
        vector<char> seen(n);
        for (int s = 0; s < n; s++)
            if (!seen[s]) {
                vector<pair<int, int>> st = {{s, 0}};
                seen[s] = 1;
                while (!st.empty()) {
                    auto &[u, i] = st.back();
                    if (i < (int)g[u].size()) {
                        int v = g[u][i++];
                        if (!seen[v])
                            seen[v] = 1, st.push_back({v, 0});
                    } else
                        order.push_back(u), st.pop_back();
                }
            }
    }
}

```

```

reverse(order.begin(), order.end());
for (int s : order)
    if (id[s] < 0) {
        id[s] = c;
        vector<int> st = {s};
        while (!st.empty()) {
            int u = st.back();
            st.pop_back();
            for (int v : rg[u])
                if (id[v] < 0)
                    id[v] = c, st.push_back(v);
        }
        c++;
    }
};

struct LowLink {
    int n, timer = 0;
    vector<pair<int, int>> e;
    vector<vector<pair<int, int>>> g;
    vector<int> tin, low, sub, parent_edge, it;
    vector<char> bridge;
    vector<vector<int>> parts;
    LowLink(int n, vector<pair<int, int>> e)
        : n(n), e(e), g(n), tin(n, -1), low(n), sub(n), parent_edge(n, -1), it(n),
          bridge(e.size()), parts(n) {
        for (int i = 0; i < (int)e.size(); i++) {
            auto [u, v] = e[i];
            g[u].push_back({v, i});
            g[v].push_back({u, i});
        }
        tin[0] = low[0] = timer++;
        sub[0] = 1;
        vector<int> st = {0};
        while (!st.empty()) {
            int u = st.back();
            if (it[u] < (int)g[u].size()) {
                auto [v, id] = g[u][it[u]++];
                if (id == parent_edge[u])
                    continue;
                if (tin[v] >= 0)
                    low[u] = min(low[u], tin[v]);
                else {
                    parent_edge[v] = id;
                    tin[v] = low[v] = timer++;
                    sub[v] = 1;
                    st.push_back(v);
                }
            } else {
                st.pop_back();
                int id = parent_edge[u];
                if (id < 0)
                    continue;
                auto [a, b] = e[id];
                int p = a ^ b ^ u;
                sub[p] += sub[u];
                low[p] = min(low[p], low[u]);
                if (low[u] > tin[p])
                    bridge[id] = 1;
                if (low[u] >= tin[p])
                    parts[p].push_back(sub[u]);
            }
        }
    }
};

```

```

    }
};
pair<vector<int>, vector<vector<int>>>
bridge_tree(int n, const vector<pair<int, int>> &e) {
    LowLink l(n, e);
    vector<int> id(n, -1);
    int c = 0;
    for (int s = 0; s < n; s++)
        if (id[s] < 0) {
            id[s] = c;
            vector<int> st = {s};
            while (!st.empty()) {
                int u = st.back();
                st.pop_back();
                for (auto [v, k] : l.g[u])
                    if (!l.bridge[k] && id[v] < 0)
                        id[v] = c, st.push_back(v);
            }
            c++;
        }
    vector<vector<int>> t(c);
    for (int i = 0; i < (int)e.size(); i++)
        if (l.bridge[i]) {
            auto [u, v] = e[i];
            u = id[u];
            v = id[v];
            t[u].push_back(v);
            t[v].push_back(u);
        }
    return {id, t};
}

struct TreeDistance {
    int n, L;
    vector<int> d;
    vector<vector<int>> up;
    TreeDistance(vector<vector<int>> t)
        : n(t.size()), L(max(1, (int)bit_width((unsigned)max(1, n)))), d(n),
          up(L, vector<int>(n)) {
        vector<pair<int, int>> st = {{0, 0}};
        while (!st.empty()) {
            auto [u, p] = st.back();
            st.pop_back();
            up[0][u] = p;
            for (int v : t[u])
                if (v != p)
                    d[v] = d[u] + 1, st.push_back({v, u});
        }
        for (int j = 1; j < L; j++)
            for (int i = 0; i < n; i++)
                up[j][i] = up[j - 1][up[j - 1][i]];
    }

    int lca(int a, int b) {
        if (d[a] < d[b])
            swap(a, b);
        int z = d[a] - d[b];
        for (int j = 0; j < L; j++)
            if (z >> j & 1)
                a = up[j][a];
        if (a == b)
            return a;
        for (int j = L - 1; j >= 0; j--)
            if (up[j][a] != up[j][b])
                a = up[j][a], b = up[j][b];
    }
};

```

```

    return up[0][a];
}
int dist(int a, int b) {
    int c = lca(a, b);
    return d[a] + d[b] - 2 * d[c];
}
};

vector<vector<int>> block_cut(int n, const vector<pair<int, int>> &e) {
    vector<vector<pair<int, int>>> g(n);
    for (int i = 0; i < (int)e.size(); i++) {
        auto [u, v] = e[i];
        g[u].push_back({v, i});
        g[v].push_back({u, i});
    }
    vector<int> tin(n, -1), low(n), edges, parent(n, -1), it(n);
    vector<vector<int>> t(n);
    int timer = 0;
    tin[0] = low[0] = timer++;
    vector<int> dfs = {0};
    while (!dfs.empty()) {
        int u = dfs.back();
        if (it[u] < (int)g[u].size()) {
            auto [v, id] = g[u][it[u]++];
            if (id == parent[u])
                continue;
            if (tin[v] < 0) {
                parent[v] = id;
                edges.push_back(id);
                tin[v] = low[v] = timer++;
                dfs.push_back(v);
            } else if (tin[v] < tin[u])
                edges.push_back(id), low[u] = min(low[u], tin[v]);
        } else {
            dfs.pop_back();
            int id = parent[u];
            if (id < 0)
                continue;
            auto [a, b] = e[id];
            int p = a ^ b ^ u;
            low[p] = min(low[p], low[u]);
            if (low[u] >= tin[p]) {
                vector<int> vs;
                while (1) {
                    int x = edges.back();
                    edges.pop_back();
                    vs.push_back(e[x].first);
                    vs.push_back(e[x].second);
                    if (x == id)
                        break;
                }
                sort(vs.begin(), vs.end());
                vs.erase(unique(vs.begin(), vs.end()), vs.end());
                int block = t.size();
                t.push_back({});
                for (int x : vs)
                    t[block].push_back(x), t[x].push_back(block);
            }
        }
    }
    return t;
}

vector<bitset<1500>> dominators(int n, const vector<pair<int, int>> &e) {
    vector<vector<int>> g(n), pred(n);

```

```

for (auto [u, v] : e)
    g[u].push_back(v), pred[v].push_back(u);
bitset<1500> reach;
reach[0] = 1;
vector<int> st = {0};
while (!st.empty()) {
    int u = st.back();
    st.pop_back();
    for (int v : g[u])
        if (!reach[v])
            reach[v] = 1, st.push_back(v);
}
vector<bitset<1500>> d(n, reach);
d[0].reset();
d[0][0] = 1;
bool change = 1;
while (change) {
    change = 0;
    for (int v = 1; v < n; v++)
        if (reach[v]) {
            bitset<1500> x = reach;
            for (int p : pred[v])
                if (reach[p])
                    x &= d[p];
            x[v] = 1;
            if (x != d[v])
                d[v] = x, change = 1;
        }
}
return d;
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v, --u, --v;
    auto d = dominators(n, e);
    vector<int> a;
    for (int v = 1; v + 1 < n; v++)
        if (d[n - 1][v])
            a.push_back(v + 1);
    cout << a.size() << '\n';
    for (int x : a)
        cout << x << ' ';
    cout << '\n';
}

```

## Python

```

import sys

sys.setrecursionlimit(1_000_000)

def scc(n, edges):
    graph = [[] for _ in range(n)]
    reverse = [[] for _ in range(n)]
    for u, v in edges:

```

```

graph[u].append(v)
reverse[v].append(u)
seen = [False] * n
order = []
for start in range(n):
    if seen[start]:
        continue
    stack = [(start, 0)]
    seen[start] = True
    while stack:
        node, index = stack[-1]
        if index < len(graph[node]):
            target = graph[node][index]
            stack[-1] = (node, index + 1)
            if not seen[target]:
                seen[target] = True
                stack.append((target, 0))
        else:
            order.append(node)
            stack.pop()
    component = [-1] * n
    count = 0
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target in reverse[node]:
                if component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    return component, count

def undirected_lowlink(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    subtree = [0] * n
    bridges = [False] * len(edges)
    separating = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        subtree[node] = 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] != -1:
                low[node] = min(low[node], tin[target])
            else:
                dfs(target, edge_id)
                subtree[node] += subtree[target]
                low[node] = min(low[node], low[target])

```

```

        if low[target] > tin[node]:
            bridges[edge_id] = True
        if low[target] >= tin[node]:
            separating[node].append(subtree[target])

    dfs(0)
    return graph, tin, low, subtree, bridges, separating

def bridge_tree(n, edges):
    graph, _, _, _, bridges, _ = undirected_lowlink(n, edges)
    component = [-1] * n
    count = 0
    for start in range(n):
        if component[start] != -1:
            continue
        component[start] = count
        stack = [start]
        while stack:
            node = stack.pop()
            for target, edge_id in graph[node]:
                if not bridges[edge_id] and component[target] == -1:
                    component[target] = count
                    stack.append(target)
        count += 1
    tree = [[] for _ in range(count)]
    for edge_id, (u, v) in enumerate(edges):
        if bridges[edge_id]:
            a, b = component[u], component[v]
            tree[a].append(b)
            tree[b].append(a)
    return component, tree

def tree_lca(tree):
    size = len(tree)
    levels = max(1, size.bit_length())
    up = [[0] * size for _ in range(levels)]
    depth = [0] * size
    stack = [(0, 0)]
    order = [0]
    while stack:
        node, parent = stack.pop()
        up[0][node] = parent
        for target in tree[node]:
            if target != parent:
                depth[target] = depth[node] + 1
                order.append(target)
                stack.append((target, node))
    for level in range(1, levels):
        previous = up[level - 1]
        up[level] = [previous[previous[node]] for node in range(size)]

    def lca(a, b):
        if depth[a] < depth[b]:
            a, b = b, a
        difference = depth[a] - depth[b]
        for level in range(levels):
            if difference >> level & 1:
                a = up[level][a]
        if a == b:
            return a
        for level in range(levels - 1, -1, -1):

```

```

        if up[level][a] != up[level][b]:
            a, b = up[level][a], up[level][b]
        return up[0][a]

def distance(a, b):
    ancestor = lca(a, b)
    return depth[a] + depth[b] - 2 * depth[ancestor]

return distance

def block_cut_tree(n, edges):
    graph = [[] for _ in range(n)]
    for edge_id, (u, v) in enumerate(edges):
        graph[u].append((v, edge_id))
        graph[v].append((u, edge_id))
    tin = [-1] * n
    low = [0] * n
    edge_stack = []
    tree = [[] for _ in range(n)]
    timer = 0

    def dfs(node, parent_edge=-1):
        nonlocal timer
        tin[node] = low[node] = timer
        timer += 1
        for target, edge_id in graph[node]:
            if edge_id == parent_edge:
                continue
            if tin[target] == -1:
                edge_stack.append(edge_id)
                dfs(target, edge_id)
                low[node] = min(low[node], low[target])
                if low[target] >= tin[node]:
                    vertices = set()
                    while True:
                        taken = edge_stack.pop()
                        vertices.update(edges[taken])
                        if taken == edge_id:
                            break
                    block = len(tree)
                    tree.append([])
                    for vertex in vertices:
                        tree[block].append(vertex)
                        tree[vertex].append(block)
            elif tin[target] < tin[node]:
                edge_stack.append(edge_id)
                low[node] = min(low[node], tin[target])

    dfs(0)
    return tree

def dominators(n, edges):
    predecessors = [[] for _ in range(n)]
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append(v)
        predecessors[v].append(u)
    reachable = 1
    stack = [0]
    while stack:
        node = stack.pop()

```

```

        for target in graph[node]:
            if not (reachable >> target & 1):
                reachable |= 1 << target
                stack.append(target)
    dom = [reachable] * n
    dom[0] = 1
    changed = True
    while changed:
        changed = False
        for node in range(1, n):
            if not (reachable >> node & 1):
                continue
            value = reachable
            for parent in predecessors[node]:
                if reachable >> parent & 1:
                    value &= dom[parent]
            value |= 1 << node
            if value != dom[node]:
                dom[node] = value
                changed = True
    return dom

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    n, m = data[:2]
    edges = [(data[i] - 1, data[i + 1] - 1) for i in range(2, 2 + 2 * m, 2)]
    dom = dominators(n, edges)
    answer = [v + 1 for v in range(1, n - 1) if dom[n - 1] >> v & 1]
    print(len(answer))
    print(*answer)

if __name__ == "__main__":
    main()

```

## Upsolving and stress testing

For A, compare SCC IDs against mutual Floyd–Warshall reachability and verify condensation degrees after deduplicating only when an exact edge count is required. For B, enumerate all assignments; test unit clauses and a variable forced both ways. For C, enumerate edge-ID permutations and verify the route uses  $m+1$  vertices; degrees without connectivity are insufficient. For D, enumerate all simple root-to-target paths on tiny graphs and intersect their vertex sets. These oracles deliberately avoid the optimized decompositions.

Carry forward the semantic distinction: SCCs compress mutual reachability, Euler stacks compress unused-edge traversal, and dominators compress a universal quantifier over rooted paths. They are not interchangeable merely because all produce DAGs or trees afterward.